LINE-BY-LINE INSTRUCTION MANUAL

Ultra Enterprise SDLC Lifecycle Orchestrator

FILE LEVEL INSTRUCTIONS

Create a namespace named **UltraEnterpriseSDLC**.
All enums, classes, and logic must be defined inside this namespace.

ENUM DEFINITIONS

ENUM: RiskLevel

- Declare an enumeration named **RiskLevel**
- Add the following values in this exact order:
    - Low
    - Medium
    - High
    - Critical

Purpose:
Represents the business risk associated with a requirement.

ENUM: SDLCStage

- Declare an enumeration named **SDLCStage with explicit integer values.**
- Add the following values in this exact order:
    - Backlog
    - Requirement
    - Design
    - Development
    - CodeReview
    - Testing
    - UAT
    - Deployment
    - Maintenance

Purpose:
Defines the strict lifecycle progression of a software work item.

CLASS: Requirement

Line-by-Line Instructions

1. Create a sealed class named **Requirement**
2. Declare a public integer property named **Id** with read-only access
   Purpose: uniquely identifies the requirement
3. Declare a public string property named **Title** with read-only access
   Purpose: stores the business requirement description
4. Declare a public property named **Risk** of type **RiskLevel** with read-only access
   Purpose: stores the risk classification
5. Create a constructor that accepts:
    - an integer named `id`
    - a string named `title`
    - a RiskLevel value named `risk`
6. Inside the constructor, assign all incoming values to the respective properties

---

CLASS: WorkItem

Line-by-Line Instructions

1. Create a sealed class named **WorkItem**
2. Declare a public integer property named **Id** with read-only access
3. Declare a public string property named **Name** with read-only access
4. Declare a public property named **Stage** of type **SDLCStage** with read and write access
5. Declare a public property named **DependencyIds** whose type is:
   a HashSet of type integer, with read-only access
   Purpose: stores IDs of work items that must be completed first
6. Create a constructor that accepts:
   - an integer named `id`
   - a string named `name`
   - an SDLCStage value named `stage`
7. Inside the constructor:
   - assign values to Id, Name, and Stage
   - initialize DependencyIds as an empty HashSet of type integer

---

CLASS: BuildSnapshot

Line-by-Line Instructions

1. Create a sealed class named **BuildSnapshot**
2. Declare a public string property named **Version** with read-only access
3. Declare a public DateTime property named **Timestamp** with read-only access
4. Create a constructor that accepts a string named `version`
5. Inside the constructor:
   - assign Version using the input value
   - assign Timestamp using the current system time

CLASS: AuditLog

Line-by-Line Instructions

1. Create a sealed class named **AuditLog.**
2. Declare a public DateTime property named **Time** with read-only access
3. Declare a public string property named **Action** with read-only access
4. Create a constructor that accepts a string named `action`
5. Inside the constructor:

- assign Time using the current system time
- assign Action using the input value

CLASS: QualityMetric

Line-by-Line Instructions

1. Create a sealed class named **QualityMetric**
2. Declare a public string property named **Name** with read-only access
3. Declare a public double property named **Score** with read-only access
4. Create a constructor that accepts:
   - a string named `name`
   - a double named `score`
5. Assign the values to the respective properties

CLASS: EnterpriseSDLCEngine **(CORE ENGINE)**

DATA MEMBERS (ALL TYPES WRITTEN VERBALLY)

---

1. Declare a **private List of type Requirement** named `_requirements`
   Purpose: stores all business requirements
2. Declare a **private Dictionary with key of type integer and value of type WorkItem** named `_workItemRegistry`
   Purpose: fast lookup of work items by ID
3. Declare a **private SortedDictionary with key of type SDLCStage and value of type List of WorkItem** named `_stageBoard`
   Purpose: ordered SDLC stage tracking
4. Declare a **private Queue of type WorkItem** named `_executionQueue`
   Purpose: FIFO execution control
5. Declare a **private Stack of type BuildSnapshot** named `_rollbackStack`
   Purpose: deployment rollback history

6. Declare a **private HashSet of type string** named `_uniqueTestSuites`
   Purpose: ensures test suite identifiers are unique
7. Declare a **private LinkedList of type AuditLog** named `_auditLedger`
   Purpose: immutable audit history
8. Declare a **private SortedList with key of type double and value of type QualityMetric** named `_releaseScoreboard`
   Purpose: ranking releases by quality score
9. Declare a private integer named `_requirementCounter`
10. Declare a private integer named `_workItemCounter`

CONSTRUCTOR: EnterpriseSDLCEngine

---

1. Create a constructor with no parameters
2. Initialize `_requirements` as an empty List of type Requirement
3. Initialize `_workItemRegistry` as an empty Dictionary with integer keys and WorkItem values
4. Initialize `_stageBoard` as an empty SortedDictionary with SDLCStage keys and List of WorkItem values
5. For every value in SDLCStage:
   - create an empty List of type WorkItem
   - associate it with the stage key inside `_stageBoard`
6. Initialize `_executionQueue` as an empty Queue of type WorkItem
7. Initialize `_rollbackStack` as an empty Stack of type BuildSnapshot
8. Initialize `_uniqueTestSuites` as an empty HashSet of type string
9. Initialize `_auditLedger` as an empty LinkedList of type AuditLog
10. Initialize `_releaseScoreboard` as an empty SortedList with double keys and QualityMetric values

---

METHOD INSTRUCTIONS (VERBAL TYPES ONLY)

Method: AddRequirement

Return type: void
Parameters:

- string named `title`
- RiskLevel named `risk`

Steps:

1. Create a Requirement using the current `_requirementCounter`
2. Increment `_requirementCounter`
3. Add the Requirement to the List of type Requirement
4. Create an AuditLog entry describing the addition
5. Append the AuditLog to the LinkedList of type AuditLog

Method: CreateWorkItem

Return type: WorkItem
Parameters:

- string named `name`
- SDLCStage named `stage`

Steps:

Instructions for `CreateWorkItem` Method

1. **Create a new WorkItem**
    - Use the current value of `_workItemCounter` as the unique identifier.
    - Pass the provided `name` and `stage` while creating the WorkItem.
2. **Increment the WorkItem counter**
    - Increase `_workItemCounter` by 1 so that the next WorkItem gets a new unique ID.
3. **Store the WorkItem in the registry**
    - Add the newly created WorkItem to the dictionary.
    - Use the WorkItem's integer `Id` as the key.

    - Store the corresponding `WorkItem` object as the value.

4. **Add the WorkItem to the stage board**
    - Identify the list of WorkItems associated with the given `SDLCStage`.
    - Add the new WorkItem to that list so it is tracked under the correct stage.
5. **Log the creation in the audit ledger**
    - Create an audit log entry describing that a WorkItem was created.
    - Include the WorkItem name and its stage in the log message.
    - Append this log entry to the audit ledger.
6. **Return the created WorkItem**
    - Provide the newly created WorkItem back to the caller for further use.

Method: AddDependency

Return type: void
Parameters:

- integer named `workItemId`
- integer named `dependsOnId`

Instructions for `AddDependency` Method

1. **Verify the existence of both work items**
    - Check whether a WorkItem with the given `workItemId` exists in the registry.
    - Check whether a WorkItem with the given `dependsOnId` also exists in the registry.

2. **Proceed only if both work items are valid**
    - Continue with dependency creation only when both identifiers are found.
    - If either identifier does not exist, do nothing.
3. **Add the dependency**
    - Access the WorkItem identified by `workItemId`.
    - Add `dependsOnId` to its list of dependency identifiers.
    - This establishes that the current WorkItem depends on another WorkItem.
4. **Record the dependency in the audit ledger**
    - Create an audit log entry describing the dependency relationship.
    - Include both WorkItem identifiers in the log message.
    - Append this entry to the audit ledger for tracking and traceability.

Method: PlanStage

Return type: void
Parameter:

- SDLCStage named `stage`

Step-by-Step Instructions for `PlanStage`

1. **Accept the stage to be planned**
    - Receive an `SDLCStage` value that represents the stage currently being planned.
2. **Retrieve all WorkItems for the specified stage**

- Access the stage board to obtain the list of WorkItems associated with the given stage.

3. **Filter WorkItems based on dependency rules (using LINQ)**
    - For each WorkItem in the stage:
        - Examine its list of dependency identifiers.
        - For every dependency identifier:
            - Look up the corresponding WorkItem in the registry.
            - Check the stage of that dependency WorkItem.
        - Confirm that **all dependencies are in a stage later than the current stage**.
    - Exclude any WorkItem whose dependencies do not meet this condition.
4. **Select only eligible WorkItems**
    - Identify WorkItems whose entire dependency set satisfies the dependency rule.
    - Treat these WorkItems as ready for execution.
5. **Enqueue eligible WorkItems for execution**
    - Add each eligible WorkItem to the execution queue.
    - Ensure WorkItems are queued in the order they are discovered.
6. **Record the stage planning action**
    - Create an audit log entry indicating that the specified stage has been planned.
    - Append this entry to the audit ledger for traceability and historical tracking.
7. **Complete the planning operation**
    - Finish the method without returning any value, leaving the execution queue and audit ledger updated.

Method: ExecuteNext

Return type: void

Steps:

1. Check if the Queue of type WorkItem is empty
2. Dequeue the next WorkItem
3. Record the previous SDLC stage
4. Advance the WorkItem to the next stage
5. Update stage lists accordingly
6. Log execution in the audit ledger

Method: RegisterTestSuite

Return type: void
Parameter:

- string named `suiteId`

Steps:

1. Insert the suiteId into the HashSet of type string
2. Automatically ignore duplicates
3. Log registration

Method: DeployRelease

Return type: void
Parameter:

- string named `version`

Steps:

1. Create a BuildSnapshot

2. Push it onto the Stack of type BuildSnapshot
3. Log deployment

Method: RollbackRelease

Return type: void

Steps:

1. Verify the Stack of type BuildSnapshot is not empty
2. Pop the most recent snapshot
3. Log rollback

Method: RecordQualityMetric

Return type: void
**Parameters:

- string named `metricName`
- double named `score`

Steps:

1. Check if the score already exists in the SortedList with double keys
2. Create a QualityMetric
3. Insert it into the SortedList with QualityMetric values

Method: PrintAuditLedger

Return type: void

Steps:

1. Iterate over the LinkedList of type AuditLog
2. Print each entry in insertion order
